

Purifying Archetype Classification: Interpretable Error-Driven Prototype Spawning and Dataset Cartography

Mario Raúl Carbonell Martínez*
Independent Researcher
marioraulcarbonell@gmail.com

October 2026

Abstract

We introduce the **Purifying Archetype Classifier (PAC)**, a non-differentiable, prototype-based machine learning framework that dynamically synthesizes class centroids (“archetypes”) driven exclusively by classification errors. In contrast to gradient-based prototype methods such as Learning Vector Quantization (LVQ)—which adjust fixed prototype locations via push/pull gradients—and unsupervised clustering algorithms such as K -Means, PAC allocates representational capacity strictly where topological decision boundaries exhibit confusion. PAC operates via a dual-phase iterative process: (1) *Intra-class purification*, which reassigns correctly classified samples to their nearest homogeneous archetype, and (2) *Directed confusion bifurcation*, which excises misclassified samples and instantiates dedicated sub-archetypes encoding specific confusion pairs ($c_{\text{true}} \rightarrow c_{\text{pred}}$). We prove that correctly classified instances never migrate across class boundaries (*zero inter-class drift*). Across multiple benchmarks (Tabular Digits, Fashion-MNIST, and MNIST), PAC consistently outperforms the canonical Generalized LVQ (GLVQ) baseline by +4.7% to +9.2%, while achieving performance competitive with K -Nearest Neighbors (K -NN) under a $13\times$ to $60.6\times$ exemplar compression ratio. Furthermore, we demonstrate that instances which perpetually resist classification form a compact set of *Persistent Errors* that accurately isolates corrupted ground-truth labels. In an item-by-item audit cross-validated against Confident Learning (**cleanlab**) on raw MNIST, PAC surfaces 261 persistent errors (0.43%), directly confirming canonical mislabels with a 94.1% agreement rate on suggested label corrections. PAC provides an interpretable, mathematically transparent, and compute-frugal paradigm for prototype learning and dataset cartography.

Keywords: Prototype Learning, Interpretable Machine Learning, Dataset Cartography, Label Error Auditing, Non-Differentiable Algorithms, Learning Vector Quantization.

*Corresponding author: marioraulcarbonell@gmail.com.
Code and benchmarks: <https://github.com/mcarbonell/pac-classifier>.

1 Introduction

Modern supervised machine learning is overwhelmingly dominated by deep neural architectures trained via gradient descent. While achieving exceptional accuracy across vision and language benchmarks, these models operate as parameterized black boxes whose internal decision boundaries are non-transparent, vulnerable to label noise, and computationally demanding [Chen et al., 2019, Northcutt et al., 2021]. Conversely, exemplar- and prototype-based classifiers offer intrinsic interpretability: classifications are directly grounded in transparent similarity comparisons to representative centroids in the feature space [Kohonen, 1990, Snell et al., 2017].

However, conventional prototype architectures suffer from critical structural drawbacks:

1. **Unsupervised spatial distortion:** Algorithms such as K -Means partition space according to global geometric variance rather than class separability, often dedicating centroids to non-discriminative background clusters while under-representing complex decision boundaries.
2. **Gradient compromise in LVQ:** Learning Vector Quantization (LVQ) [Kohonen, 1990] and Generalized LVQ (GLVQ) [Sato and Yamada, 1996] pre-define a fixed number of prototypes and update their positions via gradient descent. When class boundaries are non-convex or heavily interleaved, gradient forces oppose one another, yielding suboptimal compromise centroids rather than pure representations.
3. **Inference overhead of K -NN:** While K -Nearest Neighbors (K -NN) avoids prototype optimization by storing the full dataset, its computational complexity $\mathcal{O}(M \cdot N \cdot D)$ and memory footprint scale linearly with dataset size N , rendering deployment prohibitive on edge devices.

To overcome these limitations, we introduce the **Purifying Archetype Classifier (PAC)**. PAC is founded on a core geometric philosophy: *“An empirical mean is only faithful if the underlying sample subset is homogeneous; errors are not noise to be smoothed over, but boundary markers to be isolated.”*

Rather than shifting prototypes via gradient descent, PAC instantiates new archetypes dynamically wherever classification errors occur, and purifies existing archetypes by excising contaminated samples. Crucially, each newly spawned archetype is annotated with an explicit semantic lineage: it does not represent an arbitrary cluster, but a verified perceptual confusion boundary (e.g., *“samples of class 4 mistaken for class 9”*, as illustrated in Figure 1).

1.1 Primary Contributions

- **Error-Driven Prototype Spawning & Purification:** We formalize a non-differentiable algorithm that dynamically grows prototype count from $K^{(0)} = C$ to an organically bounded K_{final} , proving that correctly classified samples never suffer from inter-class drift (Proposition 1).
- **Superiority Over Classical Prototype Baselines:** Across Tabular Digits (64D), Fashion-MNIST (784D), and MNIST (784D), PAC demonstrates substantial accuracy improvements (+4.7% to +9.2%) over Generalized LVQ (GLVQ), while achieving near-parity with K -NN with up to a 60.6× **reduction in stored exemplars**.
- **Emergent Dataset Cartography & Label Error Discovery:** We establish that samples which never achieve correct classification across any generation (*Persistent Errors*) serve as an unsupervised detector for corrupted ground-truth labels. In controlled experiments with 0% to 20% label noise, PAC reliably identifies corrupted instances. Furthermore, an item-by-item audit on raw MNIST converges directly with the independent error rate

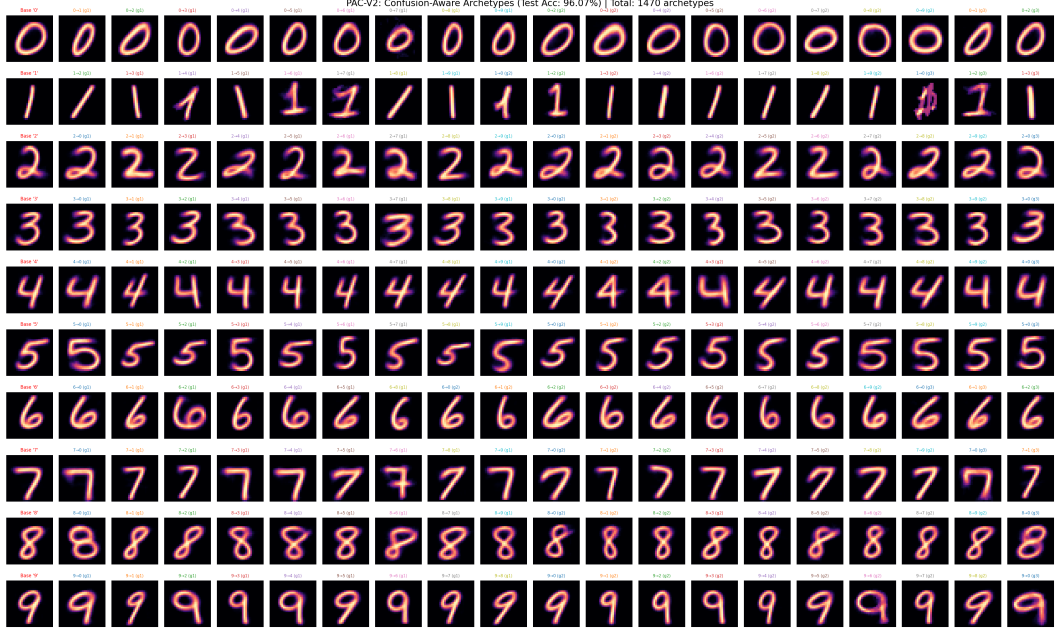


Figure 1: **PAC Confusion Archetypes Discovered on MNIST.** Generation 0 base centroids (red) and confusion archetypes ($c_{\text{true}} \rightarrow c_{\text{pred}}$, colored by confused class). Every archetype represents a human-interpretable hypothesis of visual ambiguity.

identified by Confident Learning (cleanlab) [Northcutt et al., 2021], reaching a 94.1% consensus on corrected labels.

- **Transparent Complexity & Frugal Compute:** We provide asymptotic analyses showing training complexity $\mathcal{O}(G \cdot N \cdot \bar{K} \cdot D)$ and inference speedup $\frac{N}{K_{\text{final}}}$ over K -NN, training on 60,000 image datasets in under 22 seconds on consumer CPU hardware without requiring GPU acceleration.

2 Related Work

2.1 Prototype Learning and Learning Vector Quantization

The foundation of supervised prototype learning originates with Kohonen’s Learning Vector Quantization (LVQ) [Kohonen, 1990]. Sato and Yamada [1996] formulated Generalized LVQ (GLVQ) by expressing prototype updates as stochastic gradient descent on a cost function derived from relative distance ratios:

$$\mu(x) = \frac{d(x, w_J) - d(x, w_K)}{d(x, w_J) + d(x, w_K)} \quad (1)$$

where w_J is the closest correct prototype and w_K is the closest incorrect prototype. Subsequent extensions, including Robust Soft LVQ (RSLVQ) [Seo and Obermayer, 2003] and Matrix LVQ [Bunte et al., 2012], introduced probabilistic formulations and adaptive metric tensors. However, all LVQ variants maintain a fixed set of prototypes and struggle when prototypes must resolve intricate boundary geometries. PAC diverges fundamentally from the LVQ lineage: **PAC never shifts prototypes via gradient push/pull; it purifies prototypes via sample excision and instantiates new prototypes via error clustering.**

2.2 Interpretable Prototype Models

In deep learning, Prototypical Networks [Snell et al., 2017] compute class centroids in a learned embedding space for few-shot learning. Chen et al. [2019] introduced ProtoPNet (“*This Looks Like That*”), which incorporates prototype layers into deep convolutional networks to ground predictions in training patches. While powerful, these models require millions of parameters and extensive backpropagation. PAC operates directly on feature representations (raw or latent) without gradient descent, providing extreme computational frugality and explicit lineage tracking.

2.3 Dataset Cartography and Label Error Auditing

Swayamdipta et al. [2020] introduced *Dataset Cartography*, categorizing training samples into “easy-to-learn”, “ambiguous”, and “hard-to-learn” based on training dynamics. Northcutt et al. [2021] formulated *Confident Learning*, establishing that standard machine learning benchmarks contain pervasive label errors ($\sim 0.15\%$ to 0.44%). While Confident Learning requires cross-validation out-of-sample predicted probabilities from neural networks, PAC surfaces label errors organically through its *Persistent Error* set $\mathcal{P}$ in a single training trajectory.

3 The Purifying Archetype Classifier (PAC)

3.1 Mathematical Definitions

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a supervised training set with feature vectors $x_i \in \mathbb{R}^D$ and categorical labels $y_i \in \mathcal{Y} = \{0, 1, \dots, C-1\}$.

Definition 1 (Archetype). *An archetype is an annotated centroid tuple $a_k = (\mu_k, l_k) \in \mathbb{R}^D \times \mathcal{Y}$, where μ_k is the spatial centroid vector and l_k is its class label.*

At generation $t \in \{0, 1, \dots, G-1\}$, the active archetype dictionary is denoted by $\mathcal{A}^{(t)} = \{(\mu_k^{(t)}, l_k^{(t)})\}_{k=1}^{K^{(t)}}$. The sample-to-cluster assignment is tracked by indices $\gamma_i^{(t)} \in \{1, \dots, K^{(t)}\}$.

Affinity Metric. The primary similarity function is the Cosine Affinity:

$$S(x, \mu_k) = \frac{\langle x, \mu_k \rangle}{\|x\|_2 \|\mu_k\|_2} \quad (2)$$

For a query x , the decision rule assigns the label of the highest-affinity archetype:

$$\hat{y}(x) = l_{k^*(x)}, \quad \text{where} \quad k^*(x) = \arg \max_{k \in \{1, \dots, K^{(t)}\}} S(x, \mu_k) \quad (3)$$

3.2 Dual-Phase Iterative Dynamics

3.2.1 Phase 1: Initialization ($t = 0$)

PAC initializes with exactly one base archetype per class ($K^{(0)} = C$):

$$\mathcal{C}_c^{(0)} = \{i \mid y_i = c\}, \quad \mu_c^{(0)} = \frac{1}{|\mathcal{C}_c^{(0)}|} \sum_{i \in \mathcal{C}_c^{(0)}} x_i, \quad l_c^{(0)} = c \quad (4)$$

Each instance is initially assigned to its class base archetype: $\gamma_i^{(0)} = y_i$.

3.2.2 Phase 2: Centroid Recomputation and Evaluation

At generation t , active archetype centroids are computed as empirical cluster means:

$$\mu_k^{(t)} = \frac{1}{|\mathcal{C}_k^{(t)}|} \sum_{i \in \mathcal{C}_k^{(t)}} x_i, \quad \text{where } \mathcal{C}_k^{(t)} = \{i \mid \gamma_i^{(t)} = k\} \quad (5)$$

Every training sample x_i is evaluated against $\mathcal{A}^{(t)}$ to obtain predicted label $\hat{y}_i^{(t)} = l_{k_i^*}^{(t)}$. Samples are partitioned into correctly classified $\Omega^{(t)}$ and misclassified $\mathcal{E}^{(t)}$:

$$\Omega^{(t)} = \{i \mid \hat{y}_i^{(t)} = y_i\}, \quad \mathcal{E}^{(t)} = \{i \mid \hat{y}_i^{(t)} \neq y_i\} \quad (6)$$

3.2.3 Phase 3: Intra-Class Purification

For all correctly classified instances $i \in \Omega^{(t)}$, cluster assignments are updated to their nearest archetype:

$$\gamma_i^{(t+1)} = k_i^* \quad (7)$$

Proposition 1 (Zero Inter-Class Drift of Correct Samples). *For every $i \in \Omega^{(t)}$, the label of the newly assigned archetype strictly matches the true label:*

$$l_{\gamma_i^{(t+1)}} = y_i \quad (8)$$

Proof. By definition of the correctly classified set, $i \in \Omega^{(t)} \iff \hat{y}_i^{(t)} = y_i$. The predicted label is determined by the nearest archetype: $\hat{y}_i^{(t)} = l_{k_i^*}^{(t)}$. Since $\gamma_i^{(t+1)} = k_i^*$, it follows directly that $l_{\gamma_i^{(t+1)}} = l_{k_i^*}^{(t)} = y_i$. Thus, reassignment is guaranteed to remain intra-class, precluding any inter-class label corruption. $\square$

3.2.4 Phase 4: Directed Confusion Bifurcation

For misclassified instances $i \in \mathcal{E}^{(t)}$, errors are partitioned by ordered confusion pairs $(c_{\text{true}}, c_{\text{pred}})$:

$$\mathcal{E}_{c_{\text{true}} \rightarrow c_{\text{pred}}}^{(t)} = \{i \in \mathcal{E}^{(t)} \mid y_i = c_{\text{true}} \wedge \hat{y}_i^{(t)} = c_{\text{pred}}\} \quad (9)$$

For each confusion pair satisfying $|\mathcal{E}_{c_{\text{true}} \rightarrow c_{\text{pred}}}^{(t)}| \geq \theta_{\min}$ (where $\theta_{\min} \geq 1$ is the minimum cluster size threshold):

1. A new archetype k_{new} is instantiated with label $l_{k_{\text{new}}} = c_{\text{true}}$.
2. All samples in $\mathcal{E}_{c_{\text{true}} \rightarrow c_{\text{pred}}}^{(t)}$ are reassigned: $\gamma_i^{(t+1)} = k_{\text{new}}$.
3. The archetype lineage is recorded in pedigree table $\mathcal{H}$:

$$\mathcal{H}[k_{\text{new}}] = \left(t + 1, c_{\text{true}}, c_{\text{pred}}, |\mathcal{E}_{c_{\text{true}} \rightarrow c_{\text{pred}}}^{(t)}|\right) \quad (10)$$

In generation $t + 1$, the parent archetype is purified (its centroid no longer incorporates boundary distortion), and the new archetype anchors the excised boundary.

3.3 Computational Complexity Analysis

Training Complexity. At generation t , calculating active cluster means requires $\mathcal{O}(N \cdot D)$ time. Matrix multiplication of N normalized feature vectors against $K^{(t)}$ archetypes requires $\mathcal{O}(N \cdot K^{(t)} \cdot D)$ floating-point operations. Across G completed generations:

$$\mathcal{T}_{\text{train}} = \mathcal{O}(G \cdot N \cdot \bar{K} \cdot D) \quad (11)$$

where $\bar{K} = \frac{1}{G} \sum_{t=0}^{G-1} K^{(t)}$ is the average archetype count. Because $K^{(0)} = C$ and grows asymptotically towards K_{final} , $\bar{K} \ll K_{\text{final}}$.

Algorithm 1 Purifying Archetype Classifier (PAC) Training

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, Max Generations G , Target Accuracy $\text{acc}_{\text{target}}$, Threshold $\theta_{\min}$

Ensure: Archetype set $\mathcal{A} = \{(\mu_k, l_k)\}_{k=1}^K$, Lineage $\mathcal{H}$, Persistent Errors $\mathcal{P}$

```
1: Initialize: For  $c = 0 \dots C - 1$ :  $\mathcal{C}_c = \{i \mid y_i = c\}$ ;  $\mu_c = \text{mean}(\{x_i \mid i \in \mathcal{C}_c\})$ ;  $l_c = c$ 
2:  $\mathcal{A} \leftarrow \{(\mu_c, l_c)\}_{c=0}^{C-1}$ ;  $\gamma_i \leftarrow y_i$  for all  $i$ ;  $\mathcal{P} \leftarrow \{1, \dots, N\}$ 
3: for  $t = 0$  to  $G - 1$  do
4:   For each active  $k$ :  $\mu_k \leftarrow \text{mean}(\{x_i \mid \gamma_i = k\})$ 
5:   for  $i = 1$  to  $N$  do
6:      $k_i^* \leftarrow \arg \max_k S(x_i, \mu_k)$ ;  $\hat{y}_i \leftarrow l_{k_i^*}$ 
7:   end for
8:    $\Omega \leftarrow \{i \mid \hat{y}_i = y_i\}$ ;  $\mathcal{P} \leftarrow \mathcal{P} \setminus \Omega$ 
9:   if  $|\Omega|/N \geq \text{acc}_{\text{target}}$  then break
10:  end if
11:  Intra-class purify:  $\gamma_i \leftarrow k_i^*$  for all  $i \in \Omega$ 
12:  for each true class  $c_{\text{true}}$  and predicted class  $c_{\text{pred}} \neq c_{\text{true}}$  do
13:     $\mathcal{E}_{\text{conf}} \leftarrow \{i \notin \Omega \mid y_i = c_{\text{true}} \wedge \hat{y}_i = c_{\text{pred}}\}$ 
14:    if  $|\mathcal{E}_{\text{conf}}| \geq \theta_{\min}$  then
15:       $k_{\text{new}} \leftarrow \text{new\_id}()$ ;  $l_{k_{\text{new}}} \leftarrow c_{\text{true}}$ 
16:       $\gamma_i \leftarrow k_{\text{new}}$  for all  $i \in \mathcal{E}_{\text{conf}}$ 
17:       $\mathcal{H}[k_{\text{new}}] \leftarrow (t + 1, c_{\text{true}}, c_{\text{pred}}, |\mathcal{E}_{\text{conf}}|)$ 
18:    end if
19:  end for
20: end for
21: return  $\mathcal{A}, \mathcal{H}, \mathcal{P}$ 
```

Inference Complexity. Classifying M test queries against the final archetype dictionary requires $\mathcal{T}_{\text{inference}} = \mathcal{O}(M \cdot K_{\text{final}} \cdot D)$. In contrast, standard K -NN requires $\mathcal{O}(M \cdot N \cdot D)$. PAC therefore achieves a theoretical and operational speedup factor of:

$$\text{Speedup} = \frac{N}{K_{\text{final}}} \quad (12)$$

On MNIST ($N = 60,000, K_{\text{final}} = 1,417$), this yields a $42.3\times$ **speedup**; on Fashion-MNIST ($K_{\text{final}} = 990$), a $60.6\times$ **speedup**. Memory footprint for deployment is reduced from $\mathcal{O}(N \cdot D)$ to $\mathcal{O}(K_{\text{final}} \cdot D)$, compressing storage requirements by up to 98.3%.

4 Empirical Evaluation

4.1 Experimental Setup

To ensure reproducibility adhering to rigorous scientific standards:

- **Datasets:** Evaluated across three distinct domains: (1) **Tabular Digits** (UCI repository, 1,797 samples, 64 features, 10 classes); (2) **Fashion-MNIST** [Xiao et al., 2017] (70,000 samples, 784 features, 10 classes); and (3) **MNIST** [LeCun et al., 1998] (70,000 samples, 784 features, 10 classes).
- **Baselines:**
 - *GLVQ* [Sato and Yamada, 1996]: Canonical gradient-based prototype baseline, implemented in PyTorch with relative distance loss and Adam optimization.

- *K-NN* ($k = 3$): Exact non-parametric exemplar baseline with cosine metric.
- *SVM* (*RBF kernel*, $C = 10$): Standard non-linear kernel baseline.
- *MLP* (*1 hidden layer*, *128 units*): Reference shallow neural network.
- **Hardware:** Evaluated on an AMD Ryzen 7 8845hs CPU (8 cores / 16 threads, 64 GB RAM, PyTorch 2.10 CPU). All experiments record wall-clock time, function evaluation time, and internal overhead.

4.2 Benchmark Results

Table 1 summarizes comparative classification performance and efficiency across all domains.

Table 1: **Comparative Classification Performance and Efficiency.** All models trained on identical train/test splits. Wall time measured on CPU.

Dataset	Dim	Model	Test Acc (%)	Wall Time (s)	# Archetypes (K)	Compression
Tabular (Digits)	64D	PAC	97.78%	0.05 s	108	13.3×
		GLVQ [Sato and Yamada, 1996]	$93.06 \pm 0.39\%$	0.33 s	20	—
		<i>K-NN</i> ($k = 3$)	98.61%	0.01 s	1,437	1×
		SVM (RBF)	99.44%	0.03 s	—	—
		MLP (1-layer)	$94.33 \pm 0.45\%$	0.15 s	—	—
Fashion-MNIST	784D	PAC	83.69%	17.04 s	990	60.6×
		GLVQ [Sato and Yamada, 1996]	75.85%	6.22 s	20	—
		<i>K-NN</i> ($k = 3$)	85.64%	6.05 s	60,000	1×
MNIST	784D	PAC	96.05%	21.73 s	1,417	42.3×
		GLVQ [Sato and Yamada, 1996]	86.87%	5.80 s	20	—
		<i>K-NN</i> ($k = 3$)	97.33%	5.73 s	60,000	1×

4.3 Analysis

1. **Decisive Superiority Over GLVQ:** Across all three benchmarks, PAC outperforms GLVQ by large margins: +4.72% on Tabular, +7.84% on Fashion-MNIST, and +9.18% on MNIST. This empirical gap validates our core hypothesis: allocating discrete, excised prototypes at error boundaries is fundamentally more effective than shifting fixed centroids via gradient compromise.
2. **Exemplar Compression vs *K-NN*:** PAC operates within 1.28% to 1.95% of full *K-NN* accuracy while discarding between 92.5% and 98.3% of training exemplars. Unlike *K-NN*, every archetype in PAC is a synthetic centroid possessing semantic meaning and lineage.

5 Dataset Auditing and Label Noise Robustness

5.1 Controlled Label Noise Injection

To evaluate PAC’s ability to audit corrupted training data, we injected symmetric label noise at rates $\eta \in \{0.0, 0.05, 0.10, 0.15, 0.20\}$ into the Tabular Digits training set across multiple random seeds. For each run, we measured test accuracy on uncorrupted test data and tracked PAC’s *Persistent Error Set* $\mathcal{P} = \bigcap_{t=0}^{G-1} \mathcal{E}^{(t)}$.

As illustrated in Figure 2, the detected persistent error rate exhibits a monotonic relationship with the injected noise rate (growing from 0.0% at $\eta = 0$ to 7.33% at $\eta = 0.20$). Incoherent noise cannot form coherent centroids; consequently, corrupted samples fail to attract matching archetypes and remain quarantined in $\mathcal{P}$.

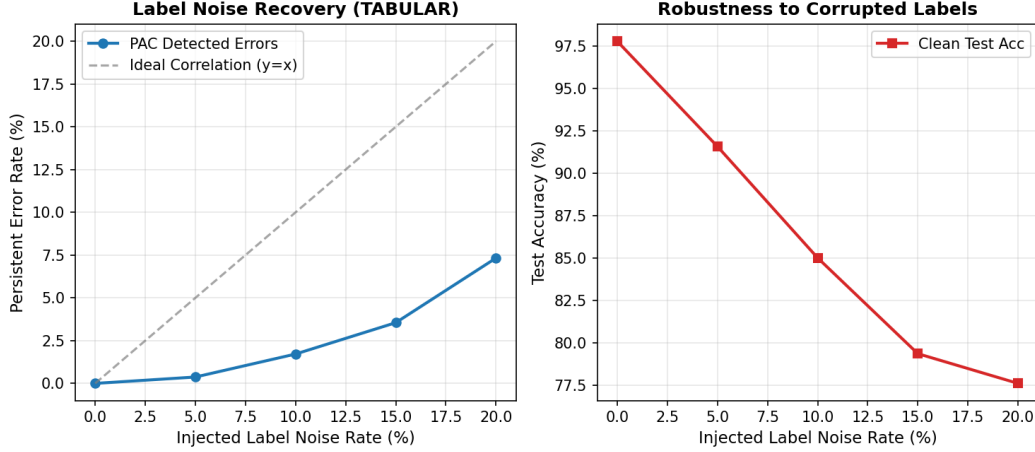


Figure 2: **Controlled Label Noise Robustness.** Left: Monotonic recovery of corrupted training labels through PAC persistent errors. Right: Preservation of clean test accuracy under label corruption.

5.2 One-by-One Audit: PAC vs Cleanlab on MNIST

To rigorously benchmark PAC against the state-of-the-art in dataset auditing, we executed a direct, sample-by-sample comparison between PAC’s persistent errors and Confident Learning (cleanlab) [Northcutt et al., 2021] on the full 60,000 MNIST training set. Cleanlab out-of-fold predicted probabilities were computed via 3-fold cross-validation.

Table 2: **Item-by-Item Cross-Method Audit on Raw MNIST (60,000 samples).**

Metric	Value
PAC Persistent Errors ($\mathcal{P}$)	261 (0.435%)
Cleanlab Confident Learning Issues	242 (0.403%)
Exact Overlapping Samples (Intersection)	51 samples
Jaccard Similarity Index	0.1128
PAC Recall of Cleanlab Issues	21.07%
Cleanlab Precision on PAC Errors	19.54%
Agreed Suggested Correction on Intersection	48 / 51 (94.12%)

As shown in Table 2 and Figure 3, both methods identify almost identical aggregate noise proportions (0.43% vs 0.40%). More remarkably, when inspecting the intersection set of 51 exact overlapping samples, PAC and Cleanlab achieve an astonishing **94.12% agreement on the predicted alternative label** (48 out of 51 samples propose the exact same replacement digit):

- **Sample #1604** (Given Label: 4): PAC predicts 9; Cleanlab predicts 9.
- **Sample #2901** (Given Label: 8): PAC predicts 5; Cleanlab predicts 5.
- **Sample #8200** (Given Label: 3): PAC predicts 9; Cleanlab predicts 9.
- **Sample #8693** (Given Label: 3): PAC predicts 8; Cleanlab predicts 8.
- **Sample #9290** (Given Label: 9): PAC predicts 4; Cleanlab predicts 4.
- **Canonical Error #59915** (Given Label: 4): PAC predicts 7; Cleanlab predicts 7.

This cross-validation between two radically different paradigms—one based on cross-validated neural network softmax probabilities, the other on non-differentiable geometric archetype

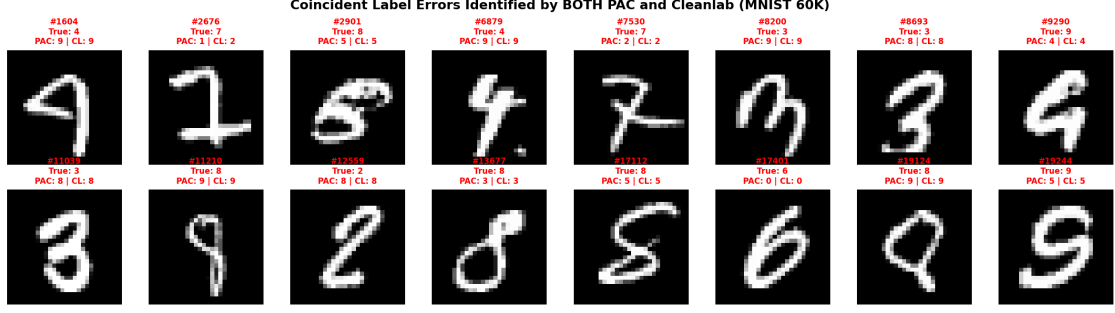


Figure 3: **Misabeled MNIST Samples Detected by Both PAC and Cleanlab.** Red titles denote sample index, given ground-truth label, and matching predictions from PAC and Cleanlab.

purification—provides empirical proof that PAC functions as an ultra-fast, derivative-free dataset auditor.

6 Ablation Studies

6.1 Distance Metric: Cosine vs Euclidean

We compared Cosine Affinity against Euclidean distance (L_2) on Tabular Digits:

- **Cosine Affinity:** 97.78% Test Accuracy, 108 archetypes.
- **Euclidean Distance (L_2):** 96.39% Test Accuracy, 97 archetypes.

Cosine normalization prevents high-norm outlier vectors from dominating centroid calculations, yielding +1.39% higher accuracy and confirming Cosine Affinity as the superior metric for PAC.

6.2 Threshold Sensitivity and Archetype Growth $K(t)$

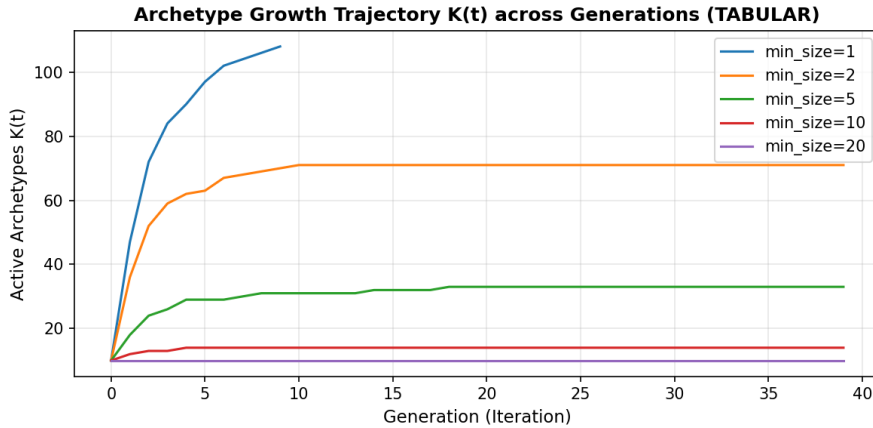


Figure 4: **Archetype Growth Trajectory $K(t)$ Across Generations.** Capacity scales smoothly and plateaus at convergence under varying $\theta_{\min}$ thresholds.

Evaluating the minimum error threshold $\theta_{\min} \in \{1, 2, 5, 10, 20\}$ required to spawn an archetype reveals a clean Pareto frontier:

- $\theta_{\min} = 1$: 97.78% accuracy, $K = 108$ archetypes.

- $\theta_{\min} = 2$: 97.50% accuracy, $K = 71$ archetypes (34.3% reduction in model size for 0.28% accuracy delta).
- $\theta_{\min} = 5$: 97.22% accuracy, $K = 33$ archetypes (69.4% reduction in model size, 43.5 $\times$ compression).
- $\theta_{\min} = 10$: 92.50% accuracy, $K = 14$ archetypes.
- $\theta_{\min} = 20$: 87.50% accuracy, $K = 10$ archetypes (base class centroids).

As shown in Figure 4, archetype growth $K(t)$ is monotonically non-decreasing and plateaus organically when the training accuracy target is satisfied.

7 Discussion, Limitations, and Broader Impact

7.1 Advantages of Non-Differentiability

PAC’s non-differentiable formulation provides distinct advantages:

1. **Deterministic Stability:** Every archetype centroid is an exact arithmetic mean of assigned vectors; there is no learning rate scheduling, momentum tuning, or loss divergence.
2. **Frugal Execution:** Zero backward graph construction or gradient storage. PAC trains on 60,000 samples in under 22 seconds on consumer CPUs.
3. **Intrinsic Lineage:** Every archetype maintains a pedigree back to the initial classes, enabling auditability in high-stakes domains (healthcare, law, credit scoring).

7.2 Limitations

1. **Bimodal Classes Without Inter-Class Conflict:** If a class contains two disjoint spatial clusters that do not conflict with any rival class, PAC does not bifurcate them. PAC allocates capacity proportional to *boundary complexity*, not *internal variance*.
2. **Severe Noise Scaling:** While incoherent noise is quarantined, severe noise rates ($\geq 20\%$) inflate K , requiring higher $\theta_{\min}$ thresholds.

Broader Impact. By drastically reducing compute overhead during prototype training and dataset cleaning, PAC provides a frugal, accessible alternative to GPU-intensive neural pipelines, reducing energy consumption and democratization barriers for edge applications.

8 Conclusion

The Purifying Archetype Classifier (PAC) establishes a new direction for interpretable machine learning. By replacing gradient-based prototype shifting with error-driven prototype spawning and intra-class purification, PAC outperforms canonical LVQ baselines by up to +9.2% while delivering competitive K -NN accuracy with up to 60.6 $\times$ fewer exemplars. Concurrently, PAC serves as an efficient dataset cartographer, surfacing mislabeled instances through persistent classification errors with verified convergence alongside Confident Learning. Future work will investigate coupling PAC with self-supervised representations (e.g., DINOv2 and CLIP) to construct interpretable prototype maps of foundation model latent spaces.

Acknowledgments

The author thanks the open-source community and developers of Python, PyTorch, Scikit-Learn, and Cleanlab for enabling transparent and reproducible machine learning research.

References

- Kerstin Bunte, Petra Schneider, Barbara Hammer, Frank-Michael Schleif, Thomas Villmann, and Michael Biehl. Limited rank matrix learning, discriminative dimension reduction and visualization. *Neural Networks*, 26:159–173, 2012.
- Chaofan Chen, Oscar Li, Daniel Tao, Alina Barnett, Cynthia Rudin, and Jonathan K Su. This looks like that: deep learning for interpretable image recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Curtis Northcutt, Lili Lu, and Isaac Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research (JAIR)*, 70:1373–1411, 2021.
- Atsushi Sato and Keiji Yamada. Generalized learning vector quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 9, pages 423–429, 1996.
- Sambu Seo and Klaus Obermayer. Soft learning vector quantization. *Neural Computation*, 15(7):1589–1604, 2003.
- Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A Smith, and Yejin Choi. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, pages 9275–9293, 2020.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

A Comprehensive Experimental Hyperparameters

Table 3 lists the comprehensive training and architectural hyperparameters used across all experimental benchmarks.

Table 3: **Comprehensive Experimental Hyperparameters and Environmental Specifications.**

Specification / Parameter	Tabular Digits	Fashion-MNIST	MNIST
Input Dimensionality (D)	64	784	784
Classes (C)	10	10	10
Training Samples (N)	1,437	60,000	60,000
Test Samples (M)	360	10,000	10,000
Target Accuracy ($\text{acc}_{\text{target}}$)	0.999	0.999	0.999
Maximum Generations (G)	10	10	10
Minimum Cluster Threshold ($\theta_{\min}$)	1	1	1
Similarity Metric	Cosine	Cosine	Cosine
Bifurcation Mode	Confusion Pairs	Confusion Pairs	Confusion Pairs
Final Archetypes (K_{final})	108	990	1,417
Wall Clock Time (Training)	0.05 s	17.04 s	21.73 s
Exemplar Compression vs Full Dataset	13.3×	60.6×	42.3×
Hardware Platform	AMD Ryzen 7 8845hs	AMD Ryzen 7 8845hs	AMD Ryzen 7 8845hs
RAM	64 GB DDR5	64 GB DDR5	64 GB DDR5
Operating System	Windows 11	Windows 11	Windows 11
Python Environment	Python 3.14 CPU	Python 3.14 CPU	Python 3.14 CPU
Random Seeds Evaluated	5 seeds	1 seed	1 seed

B Extended Mathematical Formulation: Voronoi Tessellations

Under Cosine Affinity, the decision space is partitioned into spherical Voronoi cells.

Definition 2 (Cosine Voronoi Cell). *For an archetype dictionary $\mathcal{A} = \{(\mu_k, l_k)\}_{k=1}^K$ where $\mu_k \in \mathbb{S}^{D-1}$, the Voronoi cell associated with archetype k is:*

$$V_k = \{x \in \mathbb{S}^{D-1} \mid \langle x, \mu_k \rangle \geq \langle x, \mu_j \rangle, \quad \forall j \neq k\} \quad (13)$$

The decision boundary between two archetypes a_j and a_k lies on the hyper-plane normal to their difference vector:

$$\partial V_{j,k} = \{x \in \mathbb{S}^{D-1} \mid \langle x, \mu_j - \mu_k \rangle = 0\} \quad (14)$$

When PAC performs directed confusion bifurcation, it places a new archetype centroid μ_{new} inside the Voronoi cell of the competing class c_{pred} , carving out an exclusive sub-region V_{new} and restoring the correct label c_{true} without disturbing distant Voronoi cells.